

- hw + improve
- lecture ✓
 - demo leetcode ✓
- quiz (30m)
- demo chatbot / hw gen → next time?

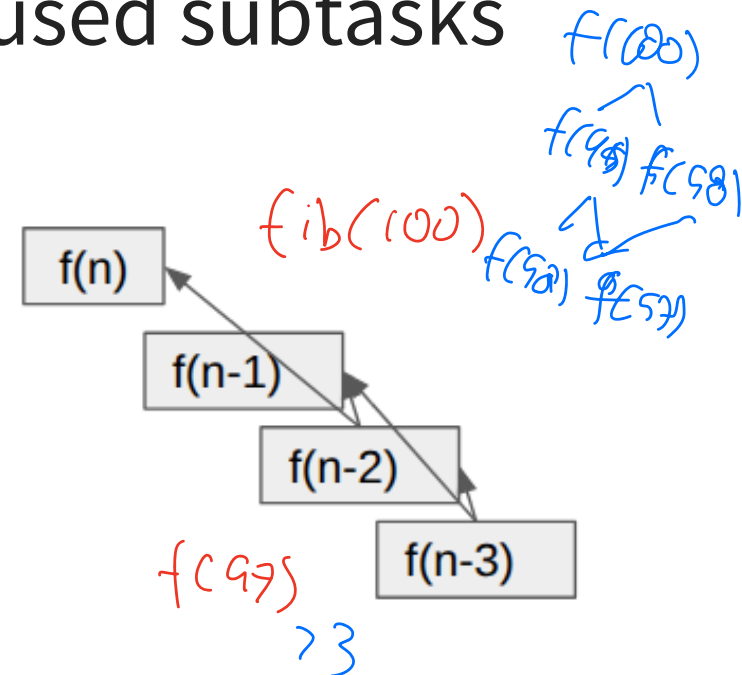
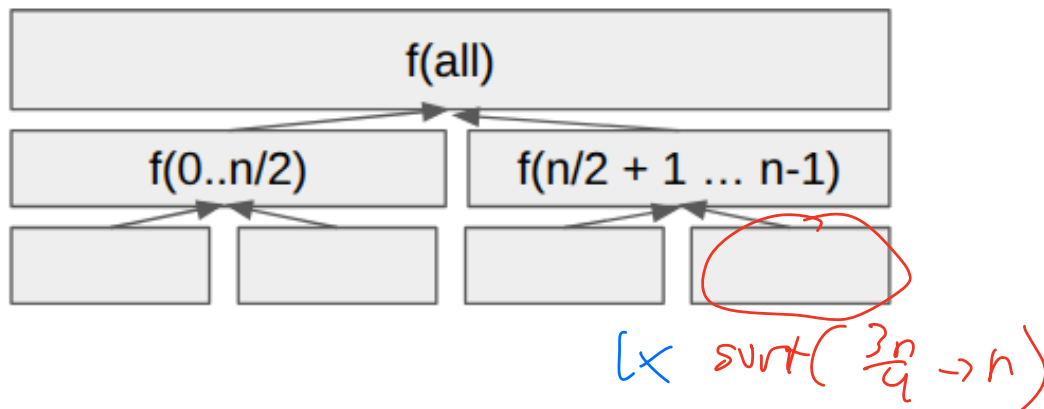
Dynamic programming (Intro)

Have I really done this before?

$$f(n) = f(n - 1) + f(n - 2)$$

How is this different?

- Divide and conquer. Independent subtasks
- Dynamic programming. Reused subtasks



Properties

$$f(\text{Big}) = \text{combine} \left(\begin{array}{l} f(\text{small1}) \\ f(\text{small2}) \end{array} \right)$$

- Optimal substructure. An optimal solution can be constructed from optimal solutions of its subproblems.
- Overlapping subproblems. A subproblem is reused several times

Implementation

↪ dynamic prog ??

- **Top down.** Call for $f(n)$ first. Recursion with cache.
- **Bottom up.** Solve for $f(0)$ first. Loops that build from base case.

↪ dynamic programming

call $f(100)$

$f(99)$ $f(98)$

= recursion + memo

Top down fibonacci

```
cache = {}
```

```
def fib(n):
```

```
    if  $n \leq 1$ : return  $n$   $\rightarrow$  base case
```

```
    if  $n$  in cache: return cache[n]  $\rightarrow$ 
```

```
    cache[n] = fib( $n-1$ ) + fib( $n-2$ )
```

```
    return cache[n]
```

$\text{fib}(3)$

3 in cache? no -

memoization

unmemoized

```
int computeCreditScore(int, d):  
    db.read("select wheel,  
            ...")
```

TTL

```
cache.set(k, v, TTL=1hr)
```

memoized

cache	
k	v
PI	120

computeCreditScore(intid):

- ① if id in cache key,
return cache[id]
- ② - compute it (slow)
- ③ save to cache

Bottom up fibonacci

```
def fib(n):  
    if n <= 1: return n  
    f = [0] * (n + 1)  
    f[1] = 1  
    for i in range(2, n + 1):  
        f[i] = f[i-1] + f[i-2]  
    return f[n]
```

i	0	1	2	3
f	0	1	1	2

Handwritten diagram showing the calculation of f[3] = f[2] + f[1] with arrows pointing to the values in the table.

runtime = $O(n)$

space = $O(n)$

fib(100)

$2^{10} = 1k$

$2^{20} = 1m$

$2^{30} = 1b$

$2^{100} = 7.7$

Q: how to reduce space complexity
from $O(n)$ to $O(1)$?

A:
$$\text{cache}[i] = \text{c}[i-1] + \text{c}[i-2]$$


$$\text{prev1} + \text{prev2}$$

save space by storing values that will be
read

House robber



House robber

```
Input: nums = [8,9,8,3]  
Output: 16
```

$(1, 3, 5, 10, 7) \rightarrow 13$ $7+5+1$
 $10+3$

Dynamic programming cookbook

1. Define subproblems. $dp[i]$ = "best" way to handle $nums[0..i]$ — *def: max value for horses $[0..i]$*
2. Find recurrence. $dp[i] = f(dp[i-1], dp[i-2], \dots)$
3. Specify order. $i = 0 \dots n-1$ or $i = n-1 \dots 0$?
4. Base case. $dp[0] = ?$

Step 1. Define subproblem

dp[i] = max value if consider nums[0..i]

nums = [8,9,8,3]

```
dp[0] = consider [8] -> 8
dp[1] = consider [8,9] -> 9
dp[2] = consider [8,9,8] -> 16
dp[3] = consider [8,9,8,3] -> 16
```

Step 2: Find recurrence

Q: Given $dp[0], \dots, dp[i - 1]$, how to find $dp[i]$?

In english: if we know how to optimally rob the first
1, 2, 3, 4 -- houses, how to optimally rob
5 houses?

Bruteforce for optimality

Be Exhaustive.

Imagine the **optimal** robber. Their plan **must** be covered by one of these cases:

- Case 1: Rob house i
- Case 2: Don't rob house i

$dp[i] = \text{max score if consider houses } [0 \dots i]$
 $dp[4] \rightarrow \max \begin{cases} \text{rob}(1, 3) \\ \text{rob}(0, 2) \end{cases}$

Case 1: Rob house i

If I rob house i , then my optimal score is ...

$$\text{nums}[i] + dp[i-2]$$

Case 2: Don't rob house i

If I don't rob house i , then my optimal score is ...

$$dp[i-1]$$

Recurrence

$$dp[i] = \max(nums[i] + dp[i - 2], dp[i - 1])$$

Recurrence correctness

I don't know the optimal solution for $dp[i]$, but I do for $dp[0 \dots i - 1]$

So, I will **brute-force** all the actions for i -th element.

The optimal solution will fall under one of the exhaustively brute-forced cases, and it too must behave **optimally** for the subproblem, else it won't be optimal

Code

```
def rob(nums):  
    n = len(nums)  
    if n == 1: return nums[0]  
  
    # Base cases  
    dp = [0] * n  
    dp[0] = nums[0]  
    dp[1] = max(nums[0], nums[1])  
  
    for i in range(2, n):  
        dp[i] = max(nums[i] + dp[i-2], dp[i-1])  
    return dp[-1]
```

Correctness

The simplest and most common form of mathematical induction infers that a statement involving a **natural number** n (that is, an integer $n \geq 0$ or 1) holds for all values of n . The proof consists of two steps:

1. The **base case** (or **initial case**): prove that the statement holds for 0 , or 1 .
2. The **induction step** (or **inductive step**, or **step case**): prove that for every n , if the statement holds for n , then it holds for $n + 1$. In other words, assume that the statement holds for some arbitrary natural number n , and prove that the statement holds for $n + 1$.

The hypothesis in the induction step, that the statement holds for a particular n , is called the **induction hypothesis** or **inductive hypothesis**. To prove the induction step, one assumes the induction hypothesis for n and then uses this assumption to prove that the statement holds for $n + 1$.

Runtime complexity

```
def rob(nums):  
    n = len(nums)  
    if n == 1: return nums[0]  
  
    dp = [0] * n  
    dp[0] = nums[0]  
    dp[1] = max(nums[0], nums[1])  
  
    for i in range(2, n):  
        dp[i] = max(nums[i] + dp[i-2], dp[i-1])  
    return dp[-1]
```

$O(n)$

Space complexity

```
def rob(nums):  
    n = len(nums)  
    if n == 1: return nums[0]  
  
    prev2 = nums[0] # was dp[i-2]  
    prev1 = max(nums[0], nums[1]) # was dp[i-1]  
  
    for i in range(2, n):  
        cur = max(nums[i] + prev2, prev1)  
        prev2, prev1 = prev1, cur  
    return prev1
```

$O(1)$

Practice Time!

① Live demo

② Your turn!

$dp[0] = -2$
 $dp[1] = 1$
 $dp[2] = 1$
 $dp[3] = 4$
 $dp[4] = 4$

Max subarray sum

$[-2, 1, -3, 4, -1, 2, 1, -5, 4]$
6

Input: $\text{int[]} \text{arr}$

Output: int maxSum

Approach 1: divide and conq
 $O(n \log n)$

Approach 2: $O(n^2)$

Div & conq



① $\downarrow$ max sum L $\downarrow$ max sum R

② $\longleftrightarrow$
find cross boundary max sum

$f(\text{int}[] \text{arr}) \rightarrow$

- ① max sum in arr
- ② max sum right
- ③ max sum on left

Approach 1:

for $l \dots$
for $r \dots$
 compute $\text{sum}[l \dots r]$
 by for loops $l \rightarrow r$

} n^3

Approach 2:

for l
 $\text{sum} = 0$
 for $r \dots$
 $\text{sum} += \text{arr}[r]$
 if $\text{sum} > \text{max}$
 $\text{max} = \text{sum}$

} $O(n^2)$

Step 1. Define subproblem

$dp[i] = \max$ subarray sum if consider $nums[0..i]$
non-empty

Step 2. Find recurrence

$dp[i] = \text{max subarray sum if consider nums}[0..i]$

- if don't take $i \rightarrow$

- if take $i \rightarrow$

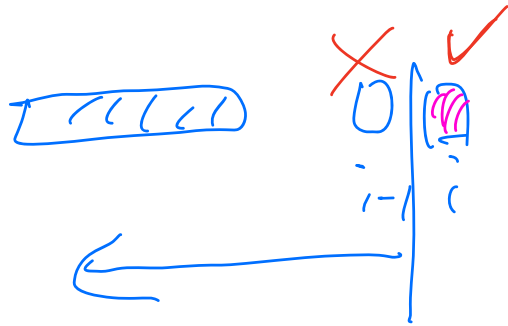
$nums[i] + dp[i-1] \rightarrow 7, \text{disgarded!!}$

Step 1. Define subproblem

$dp[i]$ = max subarray sum if consider $nums[0..i]$ and ends exactly on $nums[i]$

$[-2, 1, -3, 4, -1, 2, 1, -5, 4]$

i	dp
0	-2
1	1
2	-2
3	4



Step 2. Find recurrence

Fixed: Must take element i

- Case 1: Take element i-1 $dp[i] = nums[i] + dp[i-1]$
 - Case 2: Don't take element i-1 $dp[i] = nums[i]$
- } max

Code

Runtime: $O(n)$
space: $O(n)$

```
def maxSubArray(nums):
```

```
    n = len(nums)
```

```
    dp = [0] * n
```

```
    dp[0] = nums[0]
```

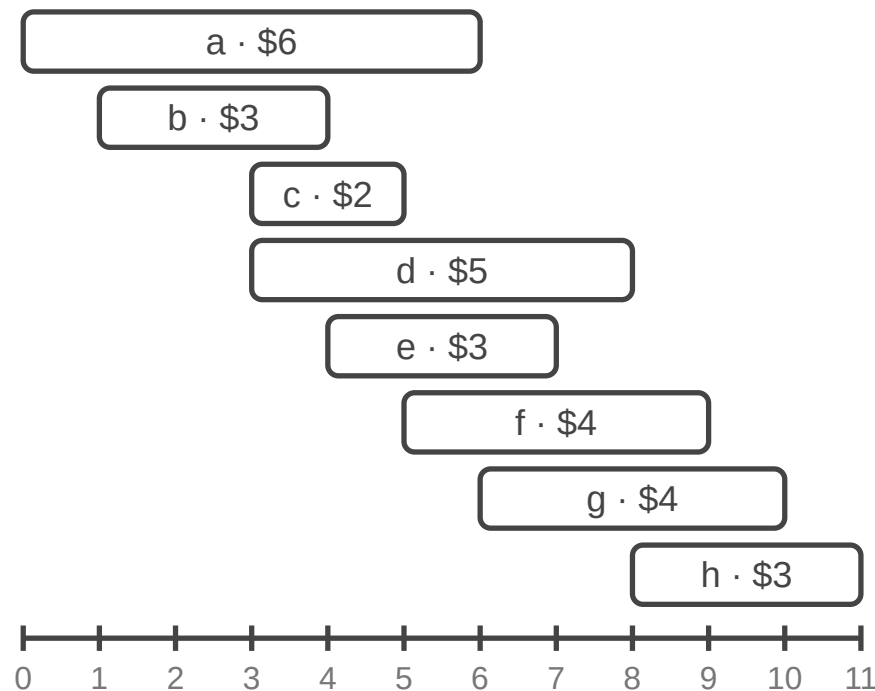
```
    for i in range(1, n):
```

```
        dp[i] = nums[i] + max(0, dp[i-1])
```

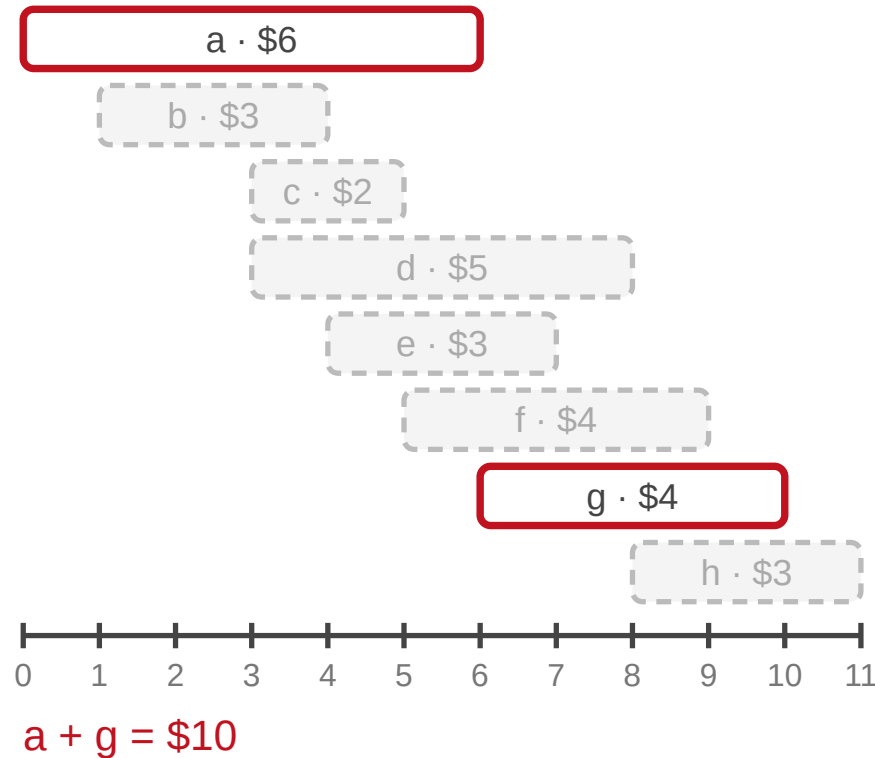
```
    return max(dp)
```

→ because $dp[n-1]$ ends exactly at $n-1$

Weighted interval scheduling



Weighted interval scheduling



Weighted interval scheduling

Input: n jobs. Job i has a start s_i , an end f_i , and a pay v_i .

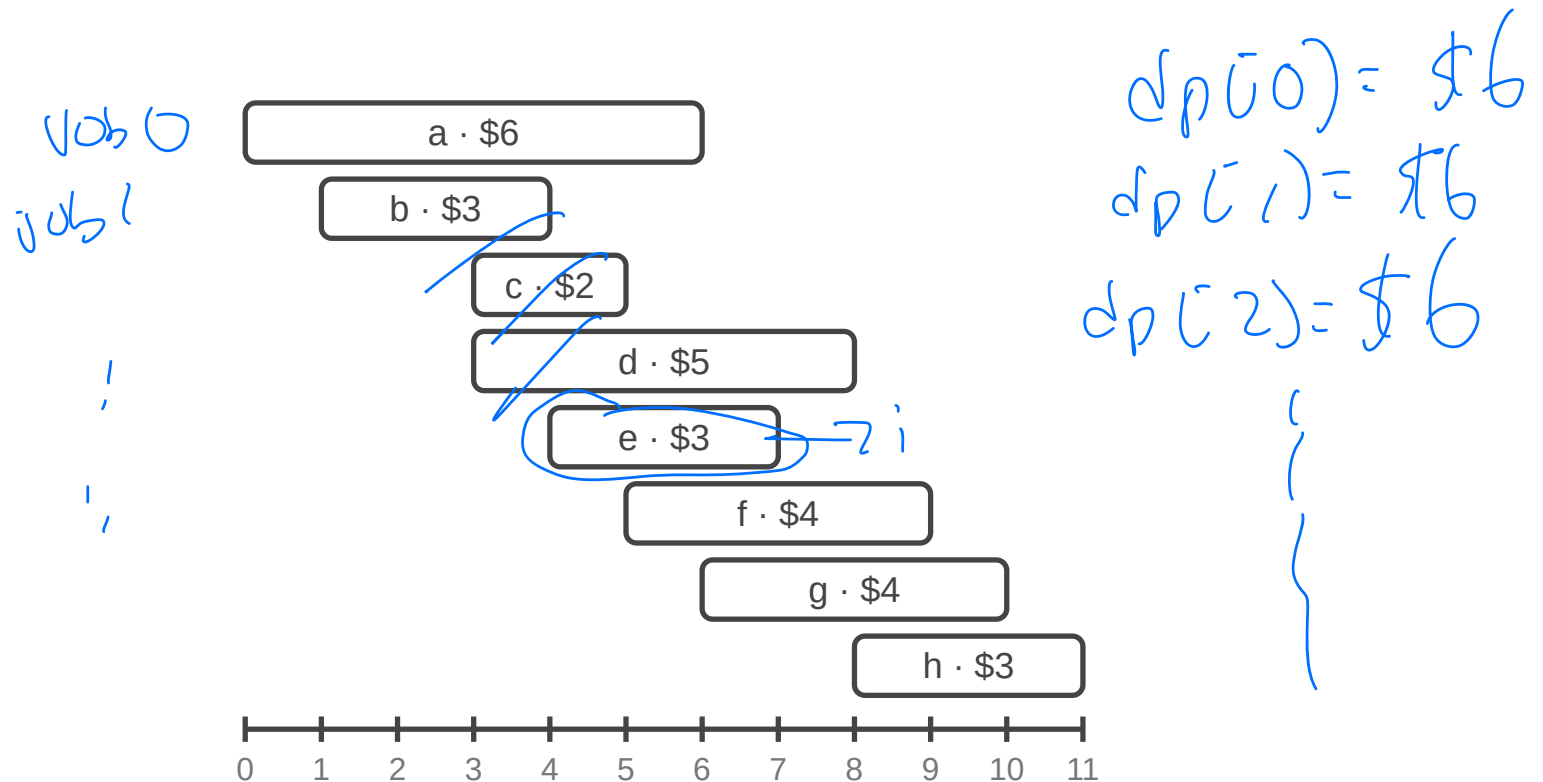
Output: The max total pay over a set of jobs, no two of which overlap.

$dp[i] =$ max total pay if have to consider jobs $(0 \dots i)$

Step 1. Define subproblems

$dp[i]$ = Max pay if we consider the first i jobs.

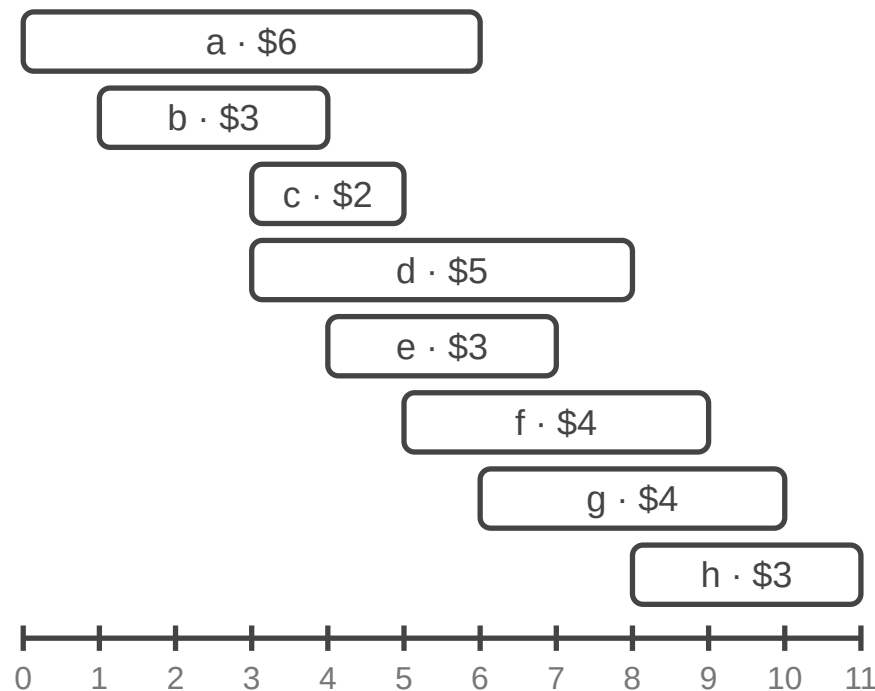
Step 1. Define subproblems



Step 2. Find recurrence

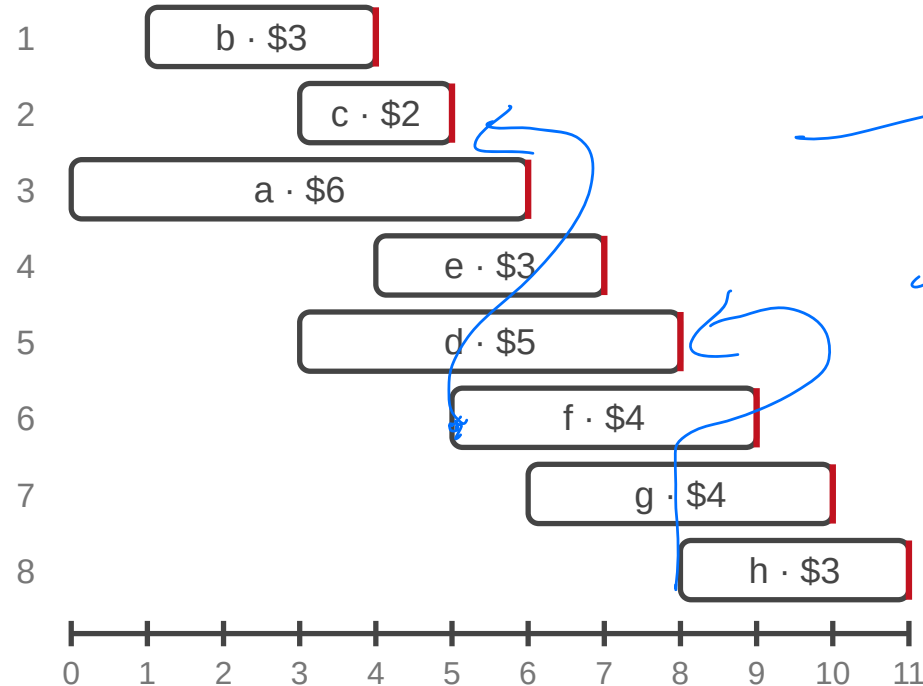
- If we skip job $i \rightarrow dp[\underline{i-1}]$ *$\rightarrow$ easy, have to do well of 1st back word.*
- If we take job $i \rightarrow ???$

Step 2. Find recurrence :(



Step 0. Preprocess

Sort the jobs by end time.



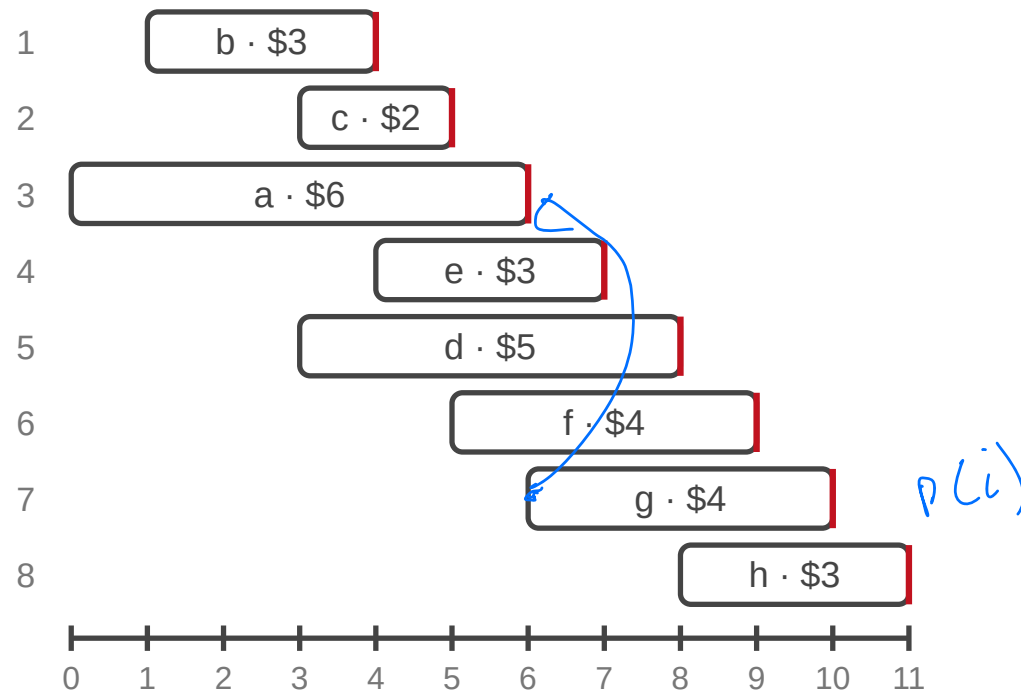
$p(i)$
the latest
compatible
job

Step 1. Define subproblems

$dp[i]$ = Max pay if we consider the **first i jobs**, sorted by
end time

$p(i)$ = the largest $j < i$ whose job **ends** at or before job i
starts

Step 1. Define subproblems

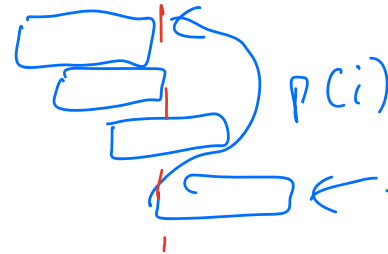


Step 2. Find recurrence

Either we take job i , or we don't.

Case 1: Skip job i

$dp[i-1]$

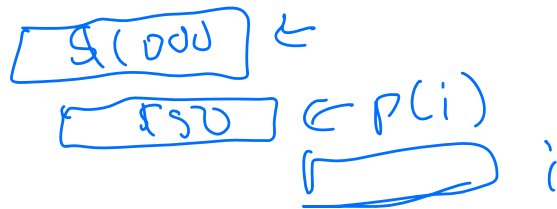


Case 2: Take job i

$$dp[i] = v_i + \underbrace{dp[p(i)]}$$

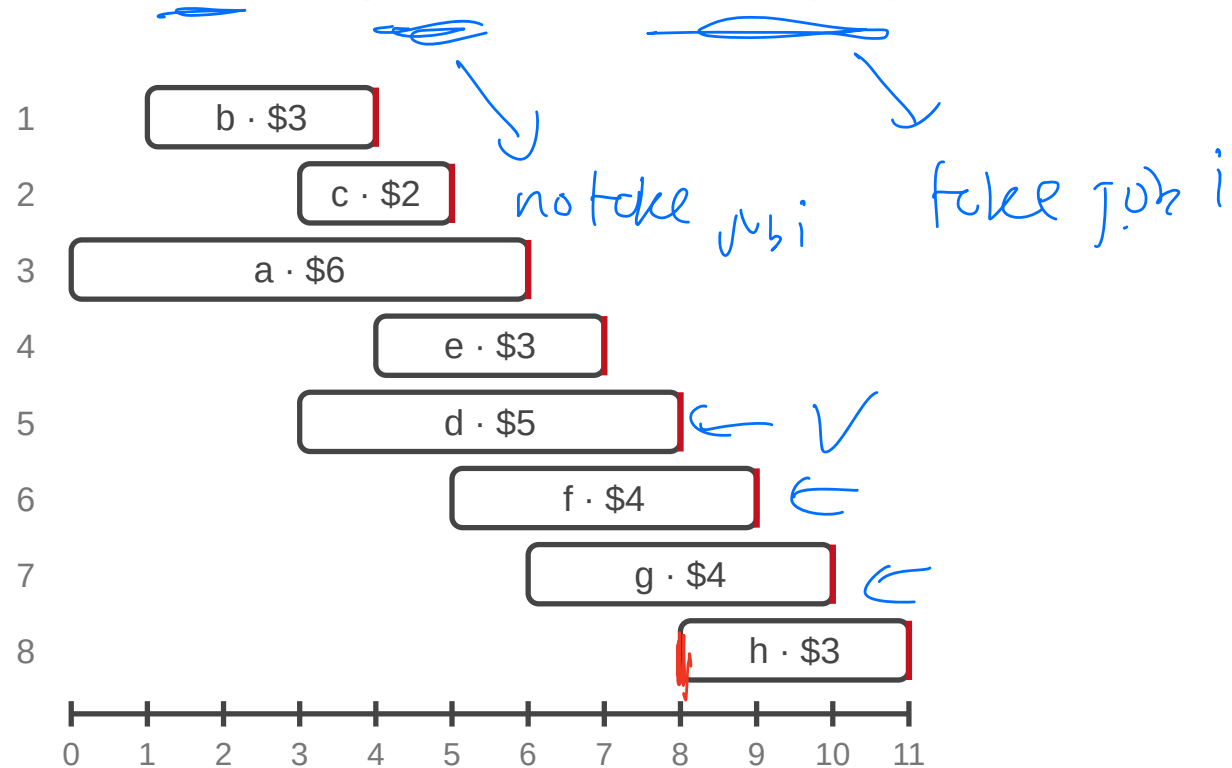
↪ max score if consider
0 ... p(i)

but you don't have to
take p(i)



Step 2. Find recurrence

$$dp[i] = \max(dp[i-1], v_i + dp[p(i)])$$



Filling the table

i	job	v_i	p(i)	dp[i]
0	--	--	--	0
1	b	3	0	3
2	c	2	0	3
..	..	..	..	..
6	f	4	2	7
7	g	4	3	10
8	h	3	5	10

Code

```
jobs.sort(key = lambda job: job.end)
Compute p[]  $\rightarrow$  pseudocode  $\rightarrow$  binary search
dp = [0] * (n + 1)

for i in range(1, n + 1):
    job = jobs[i-1]

    # Skip job i. Always allowed.
    dp[i] = dp[i-1]

    # Take job i.
    dp[i] = max(dp[i], job.pay + dp[p(i)])

return dp[n]
```

Runtime & Space complexity

Sorting: $O(n \log n)$

Finding every $p(i)$: $O(n \log n)$

Total: $O(n \log n)$

Space: $O(n)$

Recap

1. Define subproblems. $dp[i]$ = "best" way to handle $nums[0..i]$
2. Find recurrence. $dp[i] = f(dp[i-1], dp[i-2], \dots)$
3. Specify order. $i = 0 \dots n-1$ or $i = n-1 \dots 0$?
4. Base case. $dp[0] = ?$

Tweaks

- Definition might say "must include final element"
- Preprocess